ABCI 메서드 상세 분석

개요

ABCI 인터페이스는 총 **15개의 메서드**로 구성되어 있으며, 각 메서드는 특정한 목적과 호출 시점을 가지고 있습니다. 이 문서에서는 각 메서드의 파라미터, 반환값, 호출 시점을 상세히 분석합니다.

연결 유형별 메서드 분류

1. Info/Query 연결 메서드

Info

목적: 애플리케이션의 기본 정보를 조회합니다.

호출 시점:

- CometBFT 노드 시작 시
- 다른 노드와의 핸드셰이크 과정에서

요청 파라미터:

```
type InfoRequest struct {
Version  string  // CometBFT 버전
BlockVersion  uint64  // 블록 버전
P2PVersion  uint64  // P2P 프로토콜 버전
AbciVersion  string  // ABCI 버전
```

```
}
```

응답 구조:

```
type InfoResponse struct {
Data  string  // 애플리케이션 정보
Version  string  // 애플리케이션 버전
AppVersion  uint64  // 애플리케이션 버전 번호
LastBlockHeight  int64  // 마지막 블록 높이
LastBlockAppHash []byte  // 마지막 애플리케이션 해시
LanePriorities  map[string]uint32  // 레인 우선순위 (선택사항)
DefaultLane  string  // 기본 레인 (선택사항)
}
```

구현 예제:

```
func (app *MyApplication) Info(ctx context.Context, req *InfoRequest) (*InfoResp
return &InfoResponse{
Data: "MyBlockchainApp",
Version: "1.0.0",
AppVersion: 1,
LastBlockHeight: app.state.Height,
LastBlockAppHash: app.state.AppHash,
}, nil
```

```
}
```

Query

목적: 애플리케이션 상태를 쿼리합니다.

호출 시점:

- 클라이언트가 상태 조회를 요청할 때
- RPC 엔드포인트를 통한 쿼리 요청 시

요청 파라미터:

```
type QueryRequest struct {
Data []byte  // 쿼리 데이터
Path  string  // 쿼리 경로 (예: "/store/key")
Height  int64  // 쿼리할 블록 높이 (0이면 최신)
Prove  bool  // 머클 증명 포함 여부
}
```

응답 구조:

```
type QueryResponse struct {
Code  uint32  // 응답 코드 (0 = 성공)
Log  string  // 로그 메시지 (비결정론적)
Info  string  // 추가 정보 (비결정론적)
Index  int64  // 인덱스
Key []byte  // 쿼리된 키
```

```
Value []byte  // 쿼리된 값
ProofOps *ProofOps // 머클 증명
Height  int64  // 쿼리된 블록 높이
Codespace  string  // 에러 코드 네임스페이스
}
```

구현 예제:

```
func (app *MyApplication) Query(ctx context.Context, req *QueryRequest) (*QueryR
// 경로 파싱
parts := strings.Split(req.Path, "/")
if  len(parts) < 3 || parts[1] != "store" {
return &QueryResponse{Code: 1, Log: "Invalid query path"}, nil
}
key := parts[2]
value := app.state.Get(key)
return &QueryResponse{
Code: 0,
Key: []byte(key),
Value: value,
Height: req.Height,
}, nil
```

```
}
```

2. Mempool 연결 메서드

CheckTx

목적: 트랜잭션이 메모리풀에 포함될 수 있는지 검증합니다.

호출 시점:

- 새로운 트랜잭션이 메모리풀에 추가될 때
- 블록 커밋 후 메모리풀 재검증 시 (Recheck)

요청 파라미터:

```go
type CheckTxRequest struct {
Tx []byte  // 검증할 트랜잭션
Type CheckTxType // 검증 유형
}

// CheckTxType 열거형
const (
CHECK_TX_TYPE_UNKNOWN CheckTxType = 0
CHECK_TX_TYPE_RECHECK CheckTxType = 1  // 재검증
CHECK_TX_TYPE_CHECK CheckTxType = 2  // 최초 검증
)
```

응답 구조:

```go
type CheckTxResponse struct {
Code uint32 // 응답 코드 (0 = 유효)
Data []byte // 응답 데이터
Log string // 로그 메시지 (비결정론적)
Info string // 추가 정보 (비결정론적)
GasWanted int64 // 요청된 가스량
GasUsed int64 // 사용된 가스량
Events []Event // 이벤트 목록 (비결정론적)
Codespace string // 에러 코드 네임스페이스
LaneId string // 레인 ID (선택사항)
}
```

구현 예제:

```go
func (app *MyApplication) CheckTx(ctx context.Context, req *CheckTxRequest) (*Ch
// 트랜잭션 파싱
tx, err := parseTx(req.Tx)
if err != nil {
return &CheckTxResponse{
Code: 1,
Log: "Invalid transaction format",
}, nil
}
```

```go
// 기본 유효성 검사
if !isValidFormat(tx) {
return &CheckTxResponse{
Code: 2,
Log: "Invalid transaction structure",
}, nil
}
// 서명 검증
if !verifySignature(tx) {
return &CheckTxResponse{
Code: 3,
Log: "Invalid signature",
}, nil
}
// 잔액 확인 (상태 변경 없이)
if !hasSufficientBalance(tx, app.state) {
return &CheckTxResponse{
Code: 4,
Log: "Insufficient balance",
}, nil
}
return &CheckTxResponse{
Code: 0,
```

```
GasWanted: tx.GasLimit,

GasUsed: estimateGas(tx),

}, nil

}
```

3. Consensus 연결 메서드

InitChain

목적: 블록체인을 초기화합니다.

호출 시점:

- 제네시스 블록 생성 시
- 체인 시작 시 한 번만 호출

요청 파라미터:

```
type InitChainRequest struct {

Time time.Time // 제네시스 시간

ChainId  string  // 체인 ID

ConsensusParams *ConsensusParams // 합의 파라미터

Validators []ValidatorUpdate // 초기 밸리데이터 세트

AppStateBytes []byte  // 애플리케이션 초기 상태

InitialHeight  int64  // 초기 블록 높이

}
```

응답 구조:

```go
type InitChainResponse struct {
ConsensusParams *ConsensusParams // 업데이트된 합의 파라미터
Validators []ValidatorUpdate // 업데이트된 밸리데이터 세트
AppHash []byte  // 초기 애플리케이션 해시
}
```

PrepareProposal

목적: 새로운 블록 제안을 준비합니다.

호출 시점:

- 밸리데이터가 새 블록을 제안할 때
- 합의 라운드의 Propose 단계에서

요청 파라미터:

```go
type PrepareProposalRequest struct {
MaxTxBytes  int64  // 최대 트랜잭션 바이트 수
Txs [][]byte  // 포함할 트랜잭션들
LocalLastCommit ExtendedCommitInfo // 로컬 마지막 커밋 정보
Misbehavior []Misbehavior // 악의적 행동 정보
Height  int64  // 블록 높이
Time time.Time // 블록 시간
NextValidatorsHash []byte  // 다음 밸리데이터 해시
ProposerAddress []byte  // 제안자 주소
```

```
}
```

응답 구조:

```
type PrepareProposalResponse struct {
Txs [][]byte  // 블록에 포함할 최종 트랜잭션 목록
}
```

구현 예제:

```
func (app *MyApplication) PrepareProposal(ctx context.Context, req *PrepareProposal
var  selectedTxs [][]byte
var  totalBytes  int64
// 트랜잭션 선택 및 정렬
for  _, tx := range  req.Txs {
if  totalBytes+int64(len(tx)) > req.MaxTxBytes {
break
}
// 트랜잭션 유효성 재검증
if  app.isValidTx(tx) {
selectedTxs = append(selectedTxs, tx)
totalBytes += int64(len(tx))
}
}
return &PrepareProposalResponse{
```

```
Txs: selectedTxs,
}, nil
}
```

ProcessProposal

목적: 다른 밸리데이터의 블록 제안을 검증합니다.

호출 시점:

- 다른 밸리데이터로부터 블록 제안을 받았을 때
- 합의 과정에서 제안 검증 단계

요청 파라미터:

```
type ProcessProposalRequest struct {
Txs [][]byte  // 제안된 트랜잭션들
ProposedLastCommit CommitInfo // 제안된 마지막 커밋
Misbehavior []Misbehavior // 악의적 행동 정보
Hash []byte  // 제안된 블록 해시
Height  int64  // 블록 높이
Time time.Time // 블록 시간
NextValidatorsHash []byte  // 다음 밸리데이터 해시
ProposerAddress []byte  // 제안자 주소
}
```

응답 구조:

```go
type ProcessProposalResponse struct {
Status ProcessProposalStatus // 제안 처리 상태
}

// ProcessProposalStatus 열거형
const (
PROCESS_PROPOSAL_STATUS_UNKNOWN ProcessProposalStatus = 0
PROCESS_PROPOSAL_STATUS_ACCEPT ProcessProposalStatus = 1
PROCESS_PROPOSAL_STATUS_REJECT ProcessProposalStatus = 2
)
```

FinalizeBlock

목적: 합의된 블록을 최종 실행하고 상태를 업데이트합니다.

호출 시점:

- 블록이 합의에 도달했을 때
- 실제 상태 변경이 수행되는 시점

요청 파라미터:

```go
type FinalizeBlockRequest struct {
Txs [][]byte  // 실행할 트랜잭션들
DecidedLastCommit CommitInfo // 결정된 마지막 커밋
Misbehavior []Misbehavior // 악의적 행동 정보
Hash []byte  // 블록 해시
```

```
Height int64 // 블록 높이
Time time.Time // 블록 시간
NextValidatorsHash []byte // 다음 밸리데이터 해시
ProposerAddress []byte // 제안자 주소
SyncingToHeight int64 // 동기화 대상 높이
}
```

응답 구조:

```
type FinalizeBlockResponse struct {
Events []Event // 블록 이벤트들
TxResults []ExecTxResult // 각 트랜잭션 실행 결과
ValidatorUpdates []ValidatorUpdate // 밸리데이터 업데이트
ConsensusParamUpdates *ConsensusParams // 합의 파라미터 업데이트
AppHash []byte // 새로운 애플리케이션 해시
NextBlockDelay time.Duration // 다음 블록까지의 지연시간
}

type ExecTxResult struct {
Code uint32 // 실행 결과 코드
Data []byte // 결과 데이터
Log string // 로그 (비결정론적)
Info string // 추가 정보 (비결정론적)
```

```
GasWanted int64 // 요청된 가스
GasUsed int64 // 사용된 가스
Events []Event // 트랜잭션 이벤트들
Codespace string // 에러 코드 네임스페이스
}
```

구현 예제:

```
func (app *MyApplication) FinalizeBlock(ctx context.Context, req *FinalizeBlockR
var txResults []ExecTxResult
var blockEvents []Event
// 각 트랜잭션 실행
for _, txBytes := range req.Txs {
result := app.executeTx(txBytes)
txResults = append(txResults, result)
// 트랜잭션 이벤트를 블록 이벤트에 추가
blockEvents = append(blockEvents, result.Events...)
}
// 상태 업데이트
app.state.Height = req.Height
app.state.Time = req.Time
// 애플리케이션 해시 계산
appHash := app.computeAppHash()
app.state.AppHash = appHash
```

```
return &FinalizeBlockResponse{
Events: blockEvents,
TxResults: txResults,
AppHash: appHash,
}, nil
}
```

Commit

목적: 상태 변경을 영구적으로 커밋합니다.

호출 시점:

- FinalizeBlock 실행 후
- 상태를 디스크에 영구 저장할 때

요청 파라미터:

```
type CommitRequest struct {
// 파라미터 없음
}
```

응답 구조:

```
type CommitResponse struct {
RetainHeight  int64  // 보관할 최소 블록 높이
}
```

구현 예제:

```
func (app *MyApplication) Commit(ctx context.Context, req *CommitRequest) (*Comm

// 상태를 디스크에 저장

err := app.state.Save()

if  err != nil {

return  nil, err

}

// 오래된 블록 정리를 위한 높이 반환

retainHeight := app.state.Height - 1000  // 최근 1000블록만 보관

if  retainHeight < 0 {

retainHeight = 0

}

return &CommitResponse{

RetainHeight: retainHeight,

}, nil

}
```

4. Vote Extension 메서드

ExtendVote

목적: 투표에 애플리케이션별 데이터를 추가합니다.

호출 시점:

- Precommit 투표 시

- 추가 정보를 투표에 포함하고 싶을 때

요청 파라미터:

```
type ExtendVoteRequest struct {
Hash []byte  // 블록 해시
Height  int64  // 블록 높이
Time time.Time // 블록 시간
Txs [][]byte  // 트랜잭션들
ProposedLastCommit CommitInfo // 제안된 마지막 커밋
Misbehavior []Misbehavior // 악의적 행동
NextValidatorsHash []byte  // 다음 밸리데이터 해시
ProposerAddress []byte  // 제안자 주소
}
```

응답 구조:

```
type ExtendVoteResponse struct {
VoteExtension []byte  // 재생 보호되는 투표 확장
NonRpExtension []byte  // 재생 보호되지 않는 확장
}
```

VerifyVoteExtension

목적: 다른 밸리데이터의 투표 확장을 검증합니다.

요청 파라미터:

```go
type VerifyVoteExtensionRequest struct {
Hash []byte  // 블록 해시
ValidatorAddress []byte  // 밸리데이터 주소
Height  int64  // 블록 높이
VoteExtension []byte  // 재생 보호 투표 확장
NonRpVoteExtension []byte  // 비재생 보호 투표 확장
}
```

응답 구조:

```go
type VerifyVoteExtensionResponse struct {
Status VerifyVoteExtensionStatus // 검증 상태
}

const (
VERIFY_VOTE_EXTENSION_STATUS_UNKNOWN VerifyVoteExtensionStatus = 0
VERIFY_VOTE_EXTENSION_STATUS_ACCEPT VerifyVoteExtensionStatus = 1
VERIFY_VOTE_EXTENSION_STATUS_REJECT VerifyVoteExtensionStatus = 2
)
```

5. State Sync 메서드

ListSnapshots

목적: 사용 가능한 스냅샷 목록을 반환합니다.

OfferSnapshot

목적: 제공된 스냅샷을 수락할지 결정합니다.

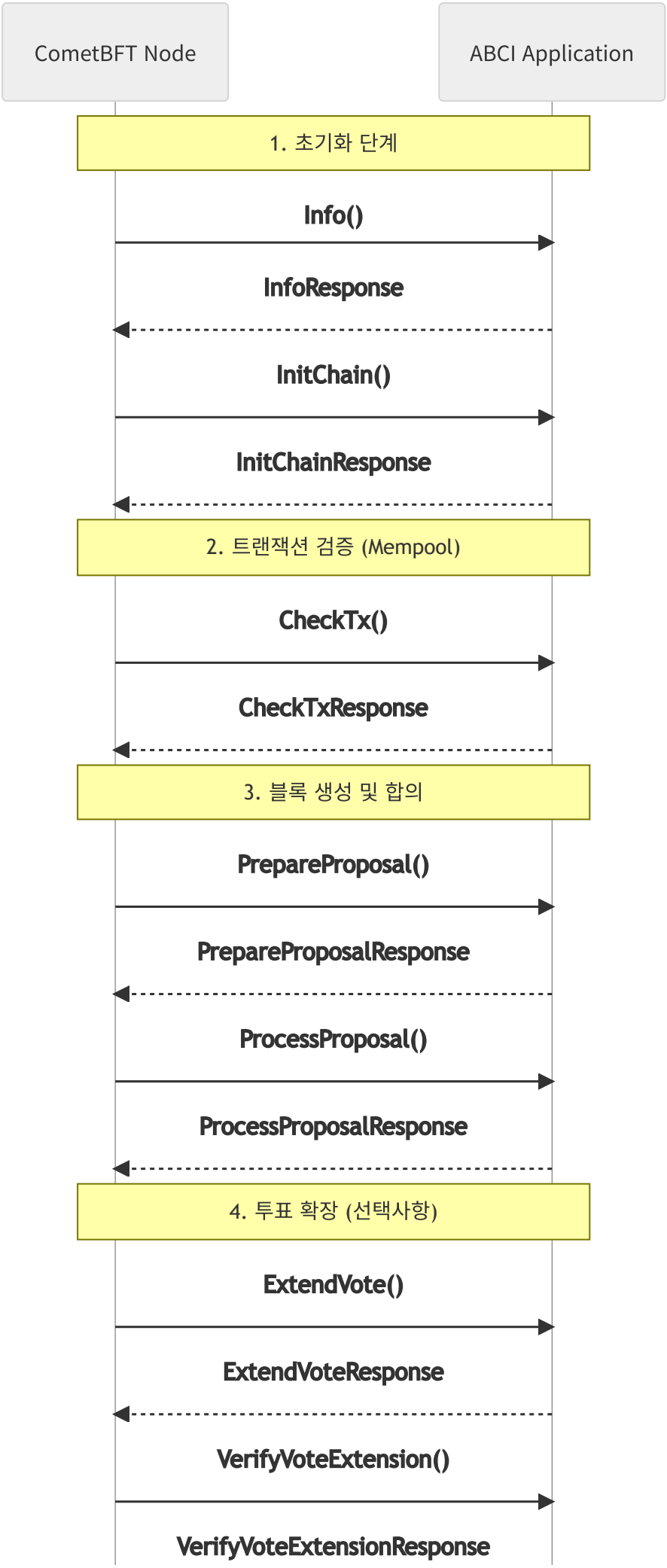
LoadSnapshotChunk

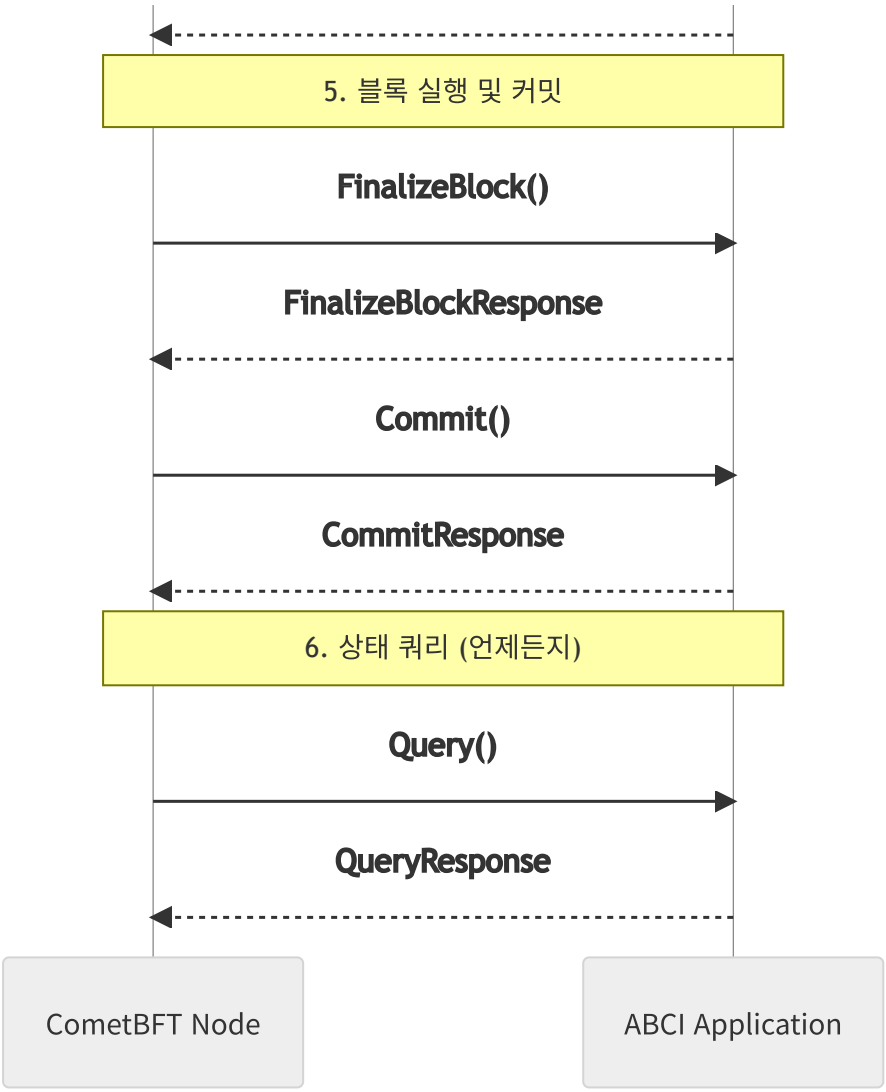
목적: 스냅샷 청크를 로드합니다.

ApplySnapshotChunk

목적: 스냅샷 청크를 적용합니다.

메서드 호출 순서와 생명주기


CometBFT Node
ABCI Application
1. 초기화 단계
Info()
InfoResponse
InitChain()
InitChainResponse
2. 트랜잭션 검증 (Mempool)
CheckTx()
CheckTxResponse
3. 블록 생성 및 합의
PrepareProposal()
PrepareProposalResponse
ProcessProposal()
ProcessProposalResponse
4. 투표 확장 (선택사항)
ExtendVote()
ExtendVoteResponse
VerifyVoteExtension()
VerifyVoteExtensionResponse



중요한 구현 고려사항

1. 결정론적 실행

- **FinalizeBlock**에서만 상태 변경
- **CheckTx**는 상태를 변경하지 않음
- 동일한 입력에 대해 항상 동일한 출력 보장

2. 가스 관리

- **CheckTx**에서 가스 추정
- **FinalizeBlock**에서 실제 가스 소비
- 가스 한도 초과 시 트랜잭션 실패

3. 이벤트 시스템

- 트랜잭션 실행 결과를 이벤트로 표현
- 클라이언트가 이벤트를 통해 상태 변경 추적
- 인덱싱 가능한 이벤트 속성 설정

4. 에러 처리

- 코드 0은 성공을 의미
- 양수 코드는 애플리케이션별 에러
- Codespace를 통한 에러 분류

이러한 메서드들의 정확한 이해와 구현은 안정적이고 효율적인 블록체인 애플리케이션 개발의 핵심입니다.